RK3576 端侧多模态系统集成与性能优化工程手册

本文覆盖第五阶段 5.1 至 5.5 的架构、生命周期、异步调度、内存链路、性能工程与系统验收。当前正在执行 5.5 长时间测试，本文件是排版验证中的工作版本，不能作为最终验收结论。

1 已验收阶段与正在执行的工作.....1

2 协作信号处理的完整源码.....1

1 已验收阶段与正在执行的工作

5.1 至 5.4 已经用户验收。5.5 已冻结 171 项资料，Qt 页面新增为可选目标，原 CLI 保留。2 线程和 4 线程的短测均完成，30 分钟混合负载正在执行。最终手册会以本轮真实结果替换本工作说明。

实际短测	输入数	成功播放	进程退出码
2 线程 180 秒输入	10	10	0
4 线程 180 秒输入	10	10	0

2 协作信号处理的完整源码

该类必须在 Qt 或 SDK 建立任何线程之前构造，使线程继承阻塞信号集。普通线程调用 sigtimedwait，不在异步信号处理器中调用互斥锁、Qt 或 join。资源先停止使用者，再销毁服务。

```
#pragma once
#include <atomic>
#include <cerrno>
#include <stdexcept>
#include <thread>
#include <pthread.h>
#include <signal.h>

// Construct before SDKs/workers: all descendant threads inherit the blocked set.
// sigtimedwait runs in an ordinary thread; there is no asynchronous signal handler.
class StopSignals {
public:
    StopSignals()
    {
```

```
        sigemptyset(&signals_);
        sigaddset(&signals_, SIGINT);
        sigaddset(&signals_, SIGTERM);
        if (pthread_sigmask(SIG_BLOCK, &signals_, &previous_) != 0)
            throw std::runtime_error("Cannot block stop signals");
        try
        {
            thread_ = std::thread([this] {
                const timespec interval{0, 100000000};
                while (!finished_.load())
                {
                    const int received = sigtimedwait(&signals_, nullptr, &interval);
                    if (received == SIGINT || received == SIGTERM)
                    {
                        int empty = 0;
                        signal_.compare_exchange_strong(empty, received);
                    }
                    else if (received < 0 && errno != EAGAIN && errno != EINTR)
                    {
                        wait_failed_.store(true);
                        break;
                    }
                }
            });
        }
        catch (...)
        {
            pthread_sigmask(SIG_SETMASK, &previous_, nullptr);
            throw;
        }
    }
    ~StopSignals()
    {
        finished_.store(true);
        if (thread_.joinable()) thread_.join();
        pthread_sigmask(SIG_SETMASK, &previous_, nullptr);
    }
    StopSignals(const StopSignals&) = delete;
    StopSignals& operator=(const StopSignals&) = delete;
    int received() const noexcept { return signal_.load(); }
    bool failed() const noexcept { return wait_failed_.load(); }
private:
    sigset_t signals_{};
    sigset_t previous_{};
    std::atomic<int> signal_{0};
    std::atomic<bool> finished_{false};
    std::atomic<bool> wait_failed_{false};
    std::thread thread_;
};
```

构造中的 pthread_sigmask 将信号转交给普通等待线程。std::atomic 发布退出请求；析构先置 finished，再 join，最后恢复原信号掩码。成员的声明顺序对应 C++逆序析构规则，不能把共享状态提前释放。